LIBRARY CATALOG SYSTEM

CSE 221: Data Structure

Submitted by:

Name	ID	Intake
Md. Abu Sayed	20255103162	55
Majarul Islam Biplob	20255103174	55
Mohammad Mehedi Hasan	20255103188	55

Supervised by:

Talimul Bari Shreshtho
Lecturer,
Department of CSE

Dept. of Computer Science & Engineering
Bangladesh University of Business and Technology(BUBT)

April 29, 2026

Table of Contents

Abstract
Chapter 1: Introduction
- 1.1 Problem Statement
- 1.2 Objectives
- 1.3 Flow of the Project
- 1.4 Organization of the Report

Chapter 2: Background
- 2.1 Existing System Analysis
- 2.2 Supporting Concepts

Chapter 3: System Analysis and Design
- 3.1 Core Data Structures
- 3.2 Data Flow
- 3.3 Data Structure Summary

Chapter 4: Implementation
- 4.1 Interface Design
- 4.2 Functional Modules
- 4.3 Backend Logic

Chapter 5: User Manual
- 5.1 System Requirements
- 5.2 User Operations

Chapter 6: Conclusion
- 6.1 Conclusion
- 6.2 Limitations
- 6.3 Future Work

References

Abstract

This project presents a console-based Library Management System developed using C++, focusing on the application of fundamental data structures. The system supports operations such as searching, borrowing, returning, and managing books efficiently.

Unlike traditional systems that rely on databases, this implementation utilizes multiple data structures including Trie, Hash Maps, Stack, Queue, and Vector. Each structure is selected based on its efficiency and suitability for specific operations. The project demonstrates how combining these structures results in an optimized and practical system.

Chapter 1: Introduction

1.1 Problem Statement

Efficient management of library data is a significant challenge. Operations such as searching books, maintaining records, and managing borrowing processes require optimized handling. Without appropriate data structures, these operations become inefficient and slow.

1.2 Objectives

The main objectives of this project are:

- To demonstrate the practical use of multiple data structures
- To implement efficient searching using Trie
- To manage user and book data using Hash Maps
- To implement undo functionality using Stack
- To ensure fairness in waitlists using Queue

1.3 Flow of the Project

The system begins with a menu-driven interface. Users interact with the system by selecting options such as searching for books, borrowing, or returning. Based on user input, the system processes data using appropriate data structures.

1.4 Organization of the Report

This report is organized into six chapters. The introduction explains the problem and objectives. The background discusses related concepts. System design focuses on data structures. Implementation describes program structure. The user manual explains usage. The conclusion summarizes the project.

Chapter 2: Background

2.1 Existing System Analysis

Most modern library systems use databases and graphical interfaces. While these systems are efficient, they do not explicitly demonstrate how data structures operate internally. This project focuses on implementing core data structures directly to handle operations.

2.2 Supporting Concepts

The project is based on fundamental data structure principles including tree structures, hashing, stack operations, queue behavior, and dynamic arrays. These concepts ensure efficient data processing and retrieval.

Chapter 3: System Analysis and Design

3.1 Core Data Structures

Trie (Prefix Tree)

The Trie data structure is used for efficient prefix-based searching of book names. It allows the system to quickly locate all books matching a given prefix without scanning the entire dataset.

Time Complexity:
Insertion: O(L)
Search: O(L)
Where L is the length of the search string.

Hash Maps (unordered_map)

Hash maps are used for storing book data, user data, and borrowing records.

- Book storage: maps book ID to name and stock
- User storage: maps phone number to user credentials
- Borrow records: maps user to borrowed books

Time Complexity: O(1) average for lookup and insertion.

Vector

Vectors are used to store dynamic lists such as selected books (bucket) and search results.

Advantages:

- Dynamic resizing
- Easy indexing
- Efficient iteration

Queue

Queues are used to manage the waiting list of users for unavailable books. The First-In-First-Out principle ensures fairness.

Stack

Stacks are used to implement undo functionality. The Last-In-First-Out behavior allows the system to reverse the most recent action efficiently.

3.2 Data Flow

Search operation:
User input is processed through the Trie, and matching results are stored in a vector.

Selection operation:
Selected books are stored in a vector.

Undo operation:
The stack removes the last selected book.

Borrow operation:
Books are transferred from the selection list to user records, and stock is updated.

Return operation:
Books are removed from user records, stock is increased, and waiting users are processed using a queue.

3.3 Data Structure Summary

Feature	Data Structure	Reason
Search	Trie	Efficient prefix search
Book Storage	Hash Map	Fast lookup
User Data	Hash Map	Direct access
Selection	Vector	Dynamic storage
Waitlist	Queue	Fair processing (FIFO)
Undo	Stack	Reversible actions (LIFO)

Chapter 4: Implementation

4.1 Interface Design

The system uses a console-based interface with a menu-driven structure. The menu is repeatedly displayed until the user chooses to exit.

4.2 Functional Modules

- searchAndAdd(): Searches and selects books
- viewBucket(): Displays selected books
- checkout(): Processes borrowing
- returnBook(): Handles book return
- undoBucket(): Removes last selected book
- undo(): Reverses last borrow action

4.3 Backend Logic

The backend consists entirely of in-memory data structures. No external database or file system is used. All operations are performed using efficient algorithms and structures.

Chapter 5: User Manual

5.1 System Requirements

Hardware Requirements:
Basic computer system with minimum memory

Software Requirements:
Operating System: Windows, Linux, or macOS
Compiler: G++
Execution Environment: Terminal or Command Prompt

5.2 User Operations

Main Menu:
Displays available options such as User, Search, Borrow, Return, and Add Book

```
===== LIBRARY SYSTEM =====
1. User
2. Find
3. Borrow
4. Return
5. Add Book
0. Exit
Choice: |
```

Search:
User enters a prefix to search books

```
-- FIND --
1. Search
0. Back
Choice: 1

[Find Books]
Search: at
Searching...

Results:
1. B135 | atomic habits | 3 copies

Pick number (0 to go back): 1
Added. Nice pick.
```

Borrow:
User selects books and completes checkout

```
[Your Selection]
1. B135 | atomic habits

-- BORROW --
1. View List
2. Checkout
3. Undo Last Action
0. Back
Choice: 2

[Borrow]
Phone: 01706597322
Passcode: ***
Processing...
Done. Books are now under your name.
```

Undo:
User can remove last selected book or undo last borrow

```
-- BORROW --
1. View List
2. Checkout
3. Undo Last Action
0. Back
Choice: 3

[Undo]
Removed last selected book.
```

Return:
User returns borrowed books

```
-- RETURN --
1. Return
2. My Books
3. Undo
0. Back
Choice: |
```

Chapter 6: Conclusion

6.1 Conclusion

This project successfully demonstrates the integration of multiple data structures to build an efficient library management system. Each data structure is used based on its strengths, ensuring optimized performance and logical design.

6.2 Limitations

- No persistent storage
- Console-based interface only
- Limited validation
- No due-date management

6.3 Future Work

- Add file handling for persistent storage
- Develop graphical user interface
- Implement sorting and filtering features
- Introduce admin controls and fine system

References

Kernighan, B. W., and Ritchie, D. M. The C Programming Language
Weiss, Mark Allen. Data Structures and Algorithm Analysis
Cormen, Thomas H. Introduction to Algorithms
C++ Standard Library Documentation